

Why `pip install .` Does Not Install `uvsib` into site-packages

Diagnosis and fix

2026-05-26

Symptom

Running `pip install .` from the `uvsib` repository root reports success, but nothing importable lands in `VENV_DIR/lib/pythonX.XX/site-packages`. To make `import uvsib` work, the source tree has to be linked (symlinked) into site-packages by hand.

Root cause

There are two compounding problems: an empty package list at build time, and a package-layout mismatch versus how the code imports itself.

1. `find_packages()` returns an empty list

`setup.py` builds the package list like this:

```
from setuptools import find_packages, setup
...
setup(
    include_package_data=True,
    packages=find_packages(),
    **kwargs
)
```

`find_packages()` only discovers *regular* packages – directories that contain an `__init__.py` file. The `uvsib` repository has **zero `__init__.py` files anywhere** in the tree. Therefore:

```
>>> from setuptools import find_packages
>>> find_packages()
[]
```

With `packages=[]`, `pip install .` still builds and installs the project, but the resulting wheel contains **no Python modules at all** – only metadata (entry points, requirements, etc.). The evidence is in the generated metadata: `uvsib.egg-info/top_level.txt` is **empty**. The install “succeeds” while copying nothing into site-packages, which is exactly the observed behaviour.

2. There is no `uvsib/` package directory

Every module imports itself under a top-level `uvsib` package:

```
from uvsib.workchains.utils import load_references, load_zpe
from uvsib.codes.utils import get_element_entries
from uvsib.workflows import settings
```

But there is no `uvsib/` directory. The git checkout folder is *named* `uvsib`, and `codes/`, `workchains/`, `workflows/`, `db/` sit directly at its root – they are not nested inside a `uvsib/` package folder. So even if the build switched to `find_namespace_packages()`, it would discover `codes`, `workchains`, and `workflows` as **top-level** packages, not as `uvsib.codes`, `uvsib.workchains`, etc. The package name `uvsib` does not correspond to any directory.

Why the manual link works anyway

When the repo root is symlinked into site-packages as `uvsib`, importing works only because of **PEP 420 implicit namespace packages**. Since Python 3.3, a directory without an `__init__.py` can still be imported as a namespace package. So `site-packages/uvsib -> repo root` lets `import uvsib.workchains.co2rr` resolve one directory at a time, with no `__init__.py` required. The whole project runs on namespace-package resolution; the manual symlink simply puts the root onto the import path under the right name.

How to fix it

Listed in order of correctness.

Option A – proper package layout (recommended)

Create a `uvsib/` directory and move `codes/`, `workchains/`, `workflows/`, and `db/` into it, then add an `__init__.py` to each package directory:

```
uvsib/
  __init__.py
  codes/__init__.py
  workchains/__init__.py
  workflows/__init__.py
  db/__init__.py
```

With this layout, the unchanged `find_packages()` discovers `uvsib`, `uvsib.codes`, `uvsib.workchains`, and so on. Then `pip install .` (or `pip install -e .`) installs a working, importable package. This matches the `uvsib.*` import paths the code already uses. It is a directory move, so it is the largest change, but it is the layout the code already assumes.

Option B – no restructure: tell setuptools the root is uvsib

Keep the files where they are and map the repository root onto the `uvsib` package name in `setup.py`:

```
from setuptools import find_namespace_packages, setup

sub = find_namespace_packages(
    include=["codes*", "workchains*", "workflows*", "db*"]
)
setup(
    package_dir={"uvsib": "."},
    packages=["uvsib"] + [f"uvsib.{p}" for p in sub],
    include_package_data=True,
    **kwargs,
)
```

This installs the package without moving any files. It works, but `package_dir={"uvsib": "."}` (packaging the project root) is fragile and modern setuptools warns about it. Prefer Option A for the long term.

Editable install

`pip install -e .` is the supported equivalent of the manual symlink. It still needs a correct package list, so pair it with Option A or Option B – it does not fix the empty-packages problem on its own.

Summary

Problem	Consequence
No <code>__init__.py</code> files in the tree	<code>find_packages()</code> returns <code>[]</code>
<code>packages=[]</code> passed to <code>setup()</code>	Wheel ships metadata only; nothing installed
No <code>uvsib/</code> directory wrapping the code	Package name <code>uvsib</code> maps to no directory
Manual symlink into site-packages	Works via PEP 420 namespace packages

The fix is to give the project a real, discoverable `uvsib` package – either by restructuring into a `uvsib/` directory with `__init__.py` files (Option A) or by remapping the root in `setup.py` (Option B).